

Decision Making in Java (if, if-else, switch, break, continue, jump)

Decision-making statements in Java execute a block of code based on a condition. **Decision-making in programming is similar to decision-making in real life.** In programming, we also face situations where we want a certain block of code to be executed when some condition is fulfilled.

A programming language uses control statements to control the flow of execution of a program based on certain conditions. These are used to cause the flow of execution to advance and branch based on changes to the state of a program. Java provides several control statements to manage program flow, including:

- **Conditional Statements:** if, if-else, nested-if, if-else-if
- **Switch-Case:** For multiple fixed-value checks
- **Jump Statements:** break, continue, return

Types of Decision-Making Statements

- if
- if-else
- nested-if
- if-else-if
- switch-case
- jump - break, continue, return

1. Java if Statement

The **if statement** is the most simple decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not i.e. if a certain condition is true then a block of statements is executed otherwise not.

Syntax:

```
if(condition) {  
  
    // Statements to execute if  
  
    // condition is true  
  
}
```

Here, the **condition** after evaluation will be either true or false. if statement accepts boolean values - if the value is true then it will execute the block of statements under it. If we don't use curly braces ({}), only the next line after the if is considered as part of the if block For example,

```
if (condition) // Assume condition is true  
  
statement1; // Belongs to the if block  
  
statement2; // Does NOT belong to the if block
```

Here's what happens:

- If the condition is True statement1 executes.
- statement2 runs no matter what because it's not a part of the if block

```
// Java program to illustrate
// if statement without curly block
import java.util.*;

class IfDemo
{
    public static void main(String args[])
    {
        int i = 10;

        if (i < 15)

            // part of if block(immediate one statement
            // after if condition)
            System.out.println("Inside If block");

        // always executes as it is outside of if block
        System.out.println("10 is less than 15");

        // This statement will be executed
        // as if considers one statement by default again
        // below statement is outside of if block
        System.out.println("I am Not in if");
    }
}
```

2. Java if-else Statement

The if statement alone tells us that if a condition is true it will execute a block of statements and if the condition is false it won't. But what if we want to do something else if the condition is false? Here, comes the "else" statement. We can use the else statement with the if statement to execute a block of code when the condition is false.

Syntax:

```
if(condition){
    // Executes this block if
    // condition is true
}else{
    // Executes this block if
    // condition is false
}
```

```
// Java program to demonstrate
// the working of if-else statement
import java.util.*;

class IfElseDemo
{
    public static void main(String args[])
    {
        int i = 10;

        if (i < 15)
            System.out.println("i is smaller than 15");
        else
            System.out.println("i is greater than 15");
    }
}
```

3. Java nested-if Statement

A nested if is an if statement that is the target of another if or else. Nested if statements mean an if statement inside an if statement. Yes, java allows us to nest if statements within if statements. i.e, we can place an if statement inside another if statement.

Syntax:

```
if (condition1) {
// Executes when condition1 is true
if (condition2)
{
// Executes when condition2 is true
}
}
```

```
// Java program to demonstrate the
// working of nested-if statement
import java.util.*;

class NestedIf {
    public static void main(String args[])
    {
        int i = 10;

        if (i == 10 || i < 15) {

            // First if statement
            if (i < 15)
                System.out.println("i is smaller than 15");

            // Nested - if statement
            // Will only be executed if statement above
```

```

        // it is true
        if (i < 12)
            System.out.println(
                "i is smaller than 12 too");
        }
        else {
            System.out.println("i is greater than 15");
        }
    }
}

```

4. Java if-else-if ladder

Here, a user can decide among multiple options. The if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that 'if' is executed, and the rest of the ladder is bypassed. If none of the conditions is true, then the final else statement will be executed. There can be as many as 'else if' blocks associated with one 'if' block but only one 'else' block is allowed with one 'if' block.

Syntax:

```

if (condition1) {
    // code to be executed if condition1 is true
} else if (condition2) {
    // code to be executed if condition2 is true
} else {
    // code to be executed if all conditions are false
}

```

```

// Java program to demonstrate the
// working of if-else-if ladder
import java.util.*;

```

```

class IfElseLadder
{
    public static void main(String args[])
    {
        int i = 20;

        if (i == 10)
            System.out.println("i is 10");
        else if (i == 15)
            System.out.println("i is 15");
        else if (i == 20)

```

```

        System.out.println("i is 20");
    else
        System.out.println("i is not present");
    }
}

```

5. Java Switch Case

The switch statement is a multiway branch statement. It provides an easy way to dispatch execution to different parts of code based on the value of the expression.

Syntax:

```

switch (expression) {
    case value1:
        // code to be executed if expression == value1
        break;
    case value2:
        // code to be executed if expression == value2
        break;
    // more cases...
    default:
        // code to be executed if no cases match
}

```

```

// Java program to demonstrates the
// working of switch statements
import java.io.*;

```

```

class SwitchCase
{
    public static void main(String[] args)
    {
        int num = 20;
        switch (num) {
            case 5:
                System.out.println("It is 5");
                break;
            case 10:
                System.out.println("It is 10");
                break;

```

```

case 15:
    System.out.println("It is 15");
    break;
case 20:
    System.out.println("It is 20");
    break;
default:
    System.out.println("Not present");
}
}
}

```

6. jump Statements

Java supports three jump statements: **break**, **continue** and **return**. These three statements transfer control to another part of the program.

- **Break:** In Java, a break is majorly used for:
 - Terminate a sequence in a switch statement (discussed above).
 - To exit a loop.
 - Used as a "civilized" form of goto.
- **Continue:** Sometimes it is useful to force an early iteration of a loop. That is, you might want to continue running the loop but stop processing the remainder of the code in its body for this particular iteration. This is, in effect, a goto just past the body of the loop, to the loop's end. The continue statement performs such an action.

// Java program to demonstrates the use of

// continue in an if statement

import java.util.*;

class JumpStatement

```

{
    public static void main(String args[])
    {
        for (int i = 0; i < 10; i++)
        {

            // If the number is even
            // skip and continue
            if (i % 2 == 0)
                continue;

            // If number is odd, print it
            System.out.print(i + " ");
        }
    }
}

```

Return Statement

The return statement is used to explicitly return from a method. That is, it causes program control to transfer back to the caller of the method.

```
// Java program to demonstrate the use of return
import java.util.*;
public class ReturnStatment
{
    public static void main(String args[])
    {
        boolean t = true;
        System.out.println("Before the return.");

        if (t)
            return;

        // Compiler will bypass every statement
        // after return
        System.out.println("This won't execute.");
    }
}
```